Restricted-Spend Input Credit on the XRP Ledger

What the Ledger Enforces, and What Its History Is Worth

Veysel Guzelsoz
Wrocław University of Science and Technology
veyselguzelsoz1@gmail.com

Working note, October 2026

Abstract

Credit given to smallholder farmers as tokens that can only be spent with approved input vendors combines a ledger mechanism with an economic bet. This note examines both halves. On the XRP Ledger testnet, 13 transaction scenarios show which spending restrictions the protocol enforces and which depend on key custody, field institutions or the issuer's discretion. A budget-constrained allocation model, solved exactly and by two greedy rules, shows that a cheap heuristic is near-optimal only if it prices default risk. An individual-based simulation of a revolving fund, with parameters from Ghanaian data where they exist and explicit assumptions where they do not, then asks how much a lender gains from the repayment history the ledger accumulates. Under these assumptions the history adds 12–16% to harvest gain in five of six scenarios, custody arrangements matter more than any allocation rule, and seeing diversion on the ledger adds almost nothing to risk prediction. All farmer data are synthetic; the results are statements about a model, written to be tested in the field.

1 Motivation and questions

Smallholder farmers in sub-Saharan Africa rarely borrow: in Ghana's 2005/06 living standards survey, fewer than 20% of farming households used any credit and at most 6% used agricultural credit (Chamberlin, 2008). Restricted-spend digital credit is one response that has been described publicly: a farmer receives tokens redeemable only with registered providers of fertiliser, labour or storage, and every redemption leaves a record that can later support lending decisions (CTED, n.d.; NYUAD, 2026). This note does not describe or evaluate any deployed system; it studies the design in the abstract, with the XRP Ledger as the concrete platform. Three questions organise it:

1. **Enforcement.** Which parts of "credit that can only be spent at approved vendors" does the ledger itself enforce?
2. **Allocation.** Given a fund, a tolerance for expected loss and per-farmer default risk, how should credit be allocated, and how close do simple rules come?
3. **Information.** How much is the repayment history recorded on the ledger worth to the lender, and does seeing *how* credit was spent add to it?

Code, data and every number in this note are reproducible from the accompanying repository; tables are generated from the result files rather than typed.

2 What the XRP Ledger enforces

Setup. A fresh issuer, an approved vendor, two farmers and an unauthorised "stranger" are funded on testnet for every run. The issuer enables `RequireAuth` (only trust lines it authorises may hold the credit token), `DefaultRipple` (holders can pay each other through the issuer) and trust-line clawback. Each farmer enables `DepositAuth` and preauthorises only the issuer, so that it should be unable to receive the token from anyone else. All features used were confirmed enabled on testnet and mainnet from the on-ledger amendments object, not from documentation (XRPL, a,b).

Table 1: Escape routes tested with real testnet transactions (13 of 13 outcomes as expected). "Nobody" means no ledger rule stops the route. The hash and validated ledger index of each decisive transaction, rejected ones included, are recorded in the repository's result file.

#	Route	Observed	Enforced by
S1	farmer → approved vendor	`tesSUCCESS`	-
S2	farmer → farmer (direct payment)	`tecNO_PERMISSION`	L1: DepositAuth on receiver
S3	farmer → unauthorized account	`tecPATH_DRY`	L1: RequireAuth on issuer
S4a	ripple CRD through farmer B, trust-line flags as created	`tesSUCCESS; B CRD +10`	nobody: B's issuer line is created without NoRipple
S4b	same, after NoRipple is set on B's issuer line	`tecPATH_DRY; B CRD +0`	L1: NoRipple on both of B's lines
S4c	same, after B clears that NoRipple again	`tesSUCCESS; B CRD +10`	nobody on L1: B controls its own NoRipple flag
S5	farmer clears own DepositAuth, then farmer → farmer	`tesSUCCESS`	nobody on L1: depends on who holds the farmer's key
S6a	farmer sells credit for XRP on the open DEX, farmer buys	`tesSUCCESS; B CRD +10`	nobody on L1: offers bypass DepositAuth
S6b	farmer places an offer in the vendor-only permissioned DEX	`tecNO_PERMISSION`	L1: Permissioned DEX membership
S6c	control: vendor places an offer in the same domain	`tesSUCCESS`	-
S7	farmer and vendor settle off-ledger	`not observable`	field: audits, vendor incentives
S8a	issuer freezes farmer A's line; A → vendor	`tecPATH_DRY`	L1: issuer Freeze
S8b	issuer claws back 5 CRD from farmer A	`tesSUCCESS; A CRD -5`	L1: issuer Clawback

Findings. Table 1 records five results. *(i)* `DepositAuth` alone is not enough: a farmer can push credit into another farmer's account by rippling a self-payment through it, and the receiving farmer signs nothing (S4a). This was not the expected outcome: the farmer's trust line to the issuer was created without `NoRipple`, which must therefore be set explicitly (S4b). *(ii)* Every farmer-side protection is a flag the account holder can clear (S4c, S5): whether the restriction holds is decided by *who holds the farmer's key.* *(iii)* Offers on the open decentralised exchange deliver tokens regardless of `DepositAuth`, so credit can be sold for cash (S6a); a permissioned domain restricts its own order book, not where the token trades (S6b, S6c). *(iv)* The issuer's freeze and clawback work, but they are blunt and concentrate power in the issuer (S8). *(v)* A vendor who accepts credit and hands over cash is indistinguishable on-ledger from a legitimate sale (S7). The share of credit that "leaks" away from inputs is therefore not a property of the ledger but of the custody model and field auditing; the simulation below treats it so.

3 Allocating credit under a budget and default risk

Farmer i receives one credit level x_i from a menu including zero. With expected gain $g_i(x)$, concave in x, default probability p_i, budget B, cap L on expected loss and vendor capacities C_v, the lender solves

$$\max_{x} \sum_{i} g_i(x_i) \quad \text{s.t.} \quad \sum_{i} x_i \le B, \quad \sum_{i} p_i x_i \le L, \quad \sum_{i \in v} x_i \le C_v \;\; \forall v. \tag{1}$$

The exact optimum is found as a mixed-integer programme (HiGHS via SciPy, Virtanen et al., 2020; verified against brute-force enumeration on small instances) in about 0.5 s for 1,000 farmers. Two greedy rules raise one farmer by one level at a time: the first ranks steps by gain per unit of budget; the second by gain per unit of *both* shared resources, $\Delta g/(\Delta x/B + \lambda\, p_i \Delta x/L)$, with λ a Lagrangian-style price on the loss cap. The "searched" variant runs once for each of eleven values of λ and keeps the best; since the objective is computable by the allocator, this is part of the algorithm rather than tuning on unseen data. The "fixed" variant runs once, with $\lambda = 0.5$ chosen in advance on 100 separate instances (seeds 100–199) that are never used for evaluation.

Table 2: Expected gain as % of the exact optimum, 20 instances (placeholder parameters).

Method	Median	Range
Risk-aware greedy, λ searched	99.6	99.0–100.0
Risk-aware greedy, λ fixed	99.6	98.6–100.0
Budget-only greedy	84.0	82.5–84.9
Equal amounts	73.1	70.4–74.7
Proportional to farm size	65.4	64.4–66.8
Random, best of 200	57.3	55.5–58.9
Random, mean of 200	52.4	50.9–53.1

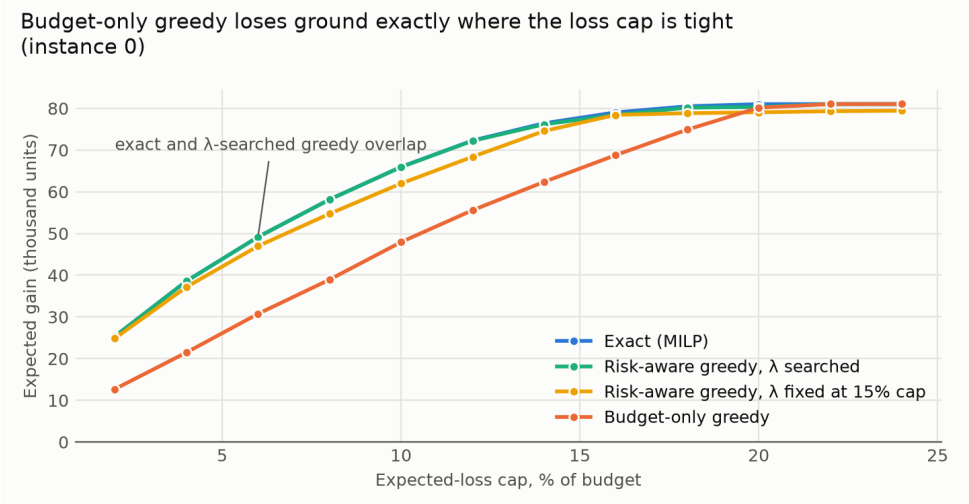


Figure 1: Gain against the loss cap. Budget-only greedy falls behind exactly where the cap is tight; a λ fixed at the 15% cap loses a few points elsewhere.

The budget-only rule reaches a median of 84.0% of the optimum, and as little as 50% when the loss cap is tight (Figure 1): it spends the scarce loss allowance on high-risk farmers. Pricing the loss cap recovers 99.6% (worst case 99.0%) with λ searched, and 99.6% (worst case 98.6%) with λ fixed from the tuning instances. The fixed value does not transfer across loss caps, however: along the frontier it falls to 94% of the optimum, as expected of a shadow price that moves with the tightness of the constraint. The practical consequence is that cheap allocation is good allocation *only with a decent estimate of* p_i — which is what a repayment history could supply.

4 A revolving-fund simulation

Model. The simulation is individual-based: heterogeneous farmers are followed over repeated seasons, but they do not optimise or interact strategically. A fund lends to 1,000 farmers in 20 villages over four seasons and relends what is repaid. Farm sizes follow the forest-zone distribution in Chamberlin (2008) (median 1.62 ha); yields and maize prices are Ghana's 2024 figures (2.60 t/ha; GH¢9,286/t; MoFA-SRID, 2025); 18% of farmers do not respond to fertiliser (Ichami et al., 2019). A loan is diverted with probability equal to the leakage rate of its custody model (5% custodial, 30% farmer-held keys; assumptions motivated by Section 2). Inputs raise the harvest by $r_i s_i\,(1-e^{-x/s_i})$, scaled by a village and an individual weather shock. Default follows

$$\Pr(\text{default}) = \sigma(\alpha + z_i + \delta\,\text{diverted} - \gamma\,\text{shock} - \kappa\,(\text{gain/amount due} - 1))\,, \tag{2}$$

where z_i is an unobserved repayment type and the last term is repayment capacity.

The return to credit is the most uncertain input. Agronomic arithmetic (18 kg grain per kg N at 2024 prices) suggests a gross return near 8.8 per cedi; on-farm trials in northern Ghana returned about 3.6 (Fosu and Dittoh 2011, as reported in Karlan et al., 2014); and a farmer-level RCT found cash grants raised harvest value by an amount not distinguishable from zero, concluding that uninsured risk, not credit, is the binding constraint (Karlan et al., 2014). That result motivates a low-return scenario; the value 1.0 is a conservative modelling choice, not an estimate from the RCT. All three are run as scenarios. The default anchor (25%) sits inside a range whose upper end, 58%, is the only Ghanaian pilot figure found (Karlan et al., 2010).

Policies. *Equal* amounts; risk-aware greedy with one pooled default rate (*no history*); the same with a logistic model of default on the ledger history (*ledger*); and the same knowing each farmer's type (*known type*, a benchmark, not attainable). Every policy runs on identical random draws, 1,000 replications per scenario.

Table 3: Total harvest gain over four seasons (million GH¢, 2024 prices), ledger gain over no history (paired), and fund left after four seasons under the ledger policy.

Return	Custody	Equal	No history	Ledger	Known type	Ledger vs none	Fund
Low, RCT-motivated (1.0)	custodial	0.58	0.68	0.79	1.26	+16.1%	0.63
Low, RCT-motivated (1.0)	self-custody	0.37	0.42	0.47	0.83	+12.0%	0.56
Trial plots (3.58)	custodial	3.43	4.08	4.62	5.20	+13.2%	0.88
Trial plots (3.58)	self-custody	1.67	1.96	2.24	3.25	+14.7%	0.65
Naive agronomic (8.8)	custodial	9.78	12.30	12.47	13.47	+1.4%	1.04
Naive agronomic (8.8)	self-custody	4.77	5.46	6.32	8.24	+15.8%	0.71

Findings. Pricing risk beats equal amounts by 14–26% in every scenario. Ledger history adds a further 12–16%, except where returns are so high that defaults are rare (1.4%): there is little to learn when almost everyone repays. Four seasons of history captures only part of what knowing farmers would be worth (under low returns, 16% of a possible 84%). Custody dominates allocation: farmer-held keys cost 40–51% of the gain under the ledger policy. And no rule rescues the fund when returns are low: it keeps 54–69% of its capital after four seasons under every policy, including the benchmark — consistent with Karlan et al. (2014): when risk binds, better targeting of credit is not enough.

5 What the ledger history tells a lender

A six-season panel is generated by lending equal amounts (a constant fund, so labels do not dry up), and each season is predicted by a model trained only on earlier seasons (100 replications).

Table 4: Out-of-time AUC in season 6, trial-plot returns.

Features	Custodial	Self-custody
None (pooled rate)	0.500	0.500
Land	0.538	0.522
+ loan and default history	0.693	0.630
+ on-ledger diversion	0.694	0.634
+ all diversion (not observable)	0.696	0.639
Known repayment type (benchmark)	0.693	0.672

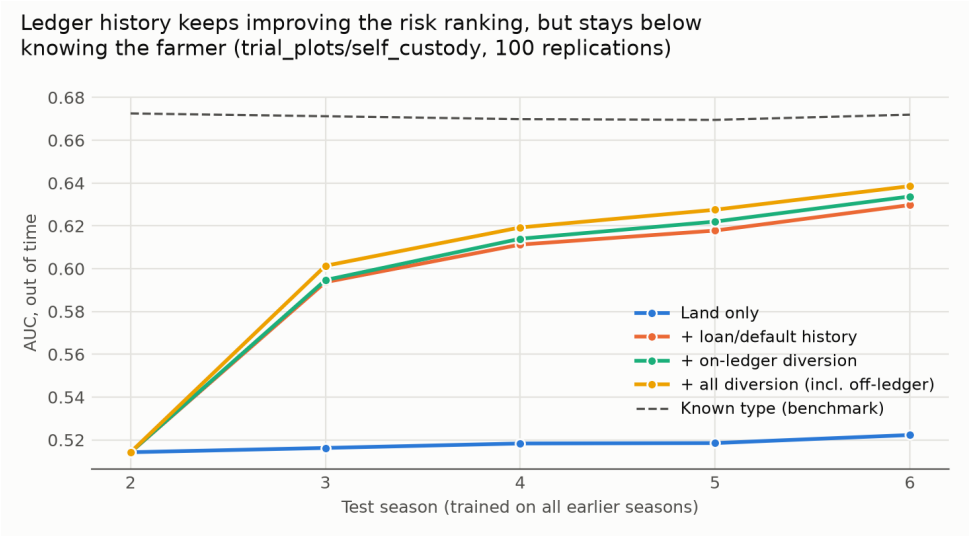


Figure 2: Out-of-time AUC by test season, self-custody.

The repayment record carries the information: with custodial wallets, five seasons of loan and default history match the known-type benchmark (Table 4). Seeing diversion adds almost nothing — even seeing *all* of it, which no ledger can (S7). In the full simulation, moving the on-ledger share of diversion from none to all changes the ledger policy's gain only from +14.1% to +15.1%. Diversion is better *prevented*, by custody, than *detected*. Two cautions: gradient boosting does worse than logistic regression on this little data (0.657 vs 0.694), and the logistic model is over-confident at the extremes (lowest decile predicted 0.15, observed 0.20; highest 0.59 vs 0.55), which matters because allocation uses p_i directly against the loss cap. Labels exist only for farmers who received a loan, so a model trained on a selective lender's book inherits its choices; randomising part of the allocation is the standard remedy.

6 Robustness

Each of fifteen unsourced assumptions was moved to a low and a high value (500 replications per point; Figure 3). No headline result changes sign except the fund's survival, which ranges from 0.62 to 1.23 times its initial capital and cannot be claimed either way. Ledger history is worth between +0.04% and +31.8%; it vanishes when defaults are rare (+0.04%) or the loss cap is loose (+0.9%) and is largest when defaults are high (+31.8%) or the cap is tight (+30.1%). The history is worth most exactly where lending is hardest. Self-custody always costs 23–69% of the gain; risk-aware

targeting always beats equal amounts (+10.8% to +37.4%). The on-ledger share of diversion is the least influential assumption. One-at-a-time ranges miss interactions; a joint design is the natural next step.

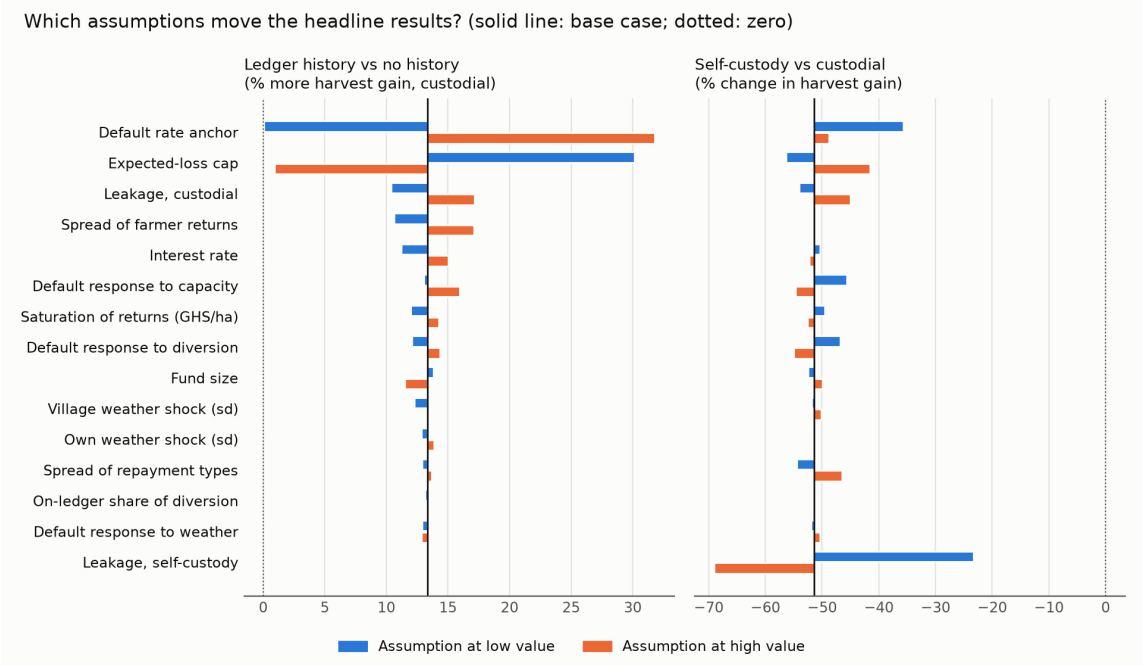


Figure 3: One-at-a-time sensitivity of two headline results.

7 How a field experiment could test this

The simulation produces hypotheses, not evidence. The two design choices it singles out — custody and allocation — can be randomised. A cluster-randomised trial would assign *villages*, not farmers, because weather is shared within a village and farmers trade with one another (spillovers through S4, S6 and S7 are local).

- **Arms.** Custodial versus farmer-held wallets, cross-randomised with history-based versus equal allocation. A third, cross-cutting arm bundling index insurance would test the Karlan et al. (2014) result directly in a token-credit setting.
- **Outcomes.** Vendor redemptions (on-ledger, near-free to measure), yields by crop cuts, repayment, and fund capital; the ledger supplies the first, the field the rest.
- **Estimand.** Average treatment effects by arm, with standard errors clustered by village; pre-registered, since the simulation already names the expected signs.

Table 5: Illustrative minimum detectable effect on the default rate (percentage points), two arms, 50 borrowers per village, baseline default 0.18, $\alpha = 0.05$, power 0.8.

Villages per arm	ICC 0.02	ICC 0.05	ICC 0.10
20	6.8	9.0	11.8
40	4.8	6.4	8.3
80	3.4	4.5	5.9

Read against the simulation (Table 5): the custody effect on default (11.2 points) is detectable with about 40 villages per arm. Detecting the allocation effect (1.4 points) in default rates would

need roughly 490 to 1,458 villages per arm for an intra-cluster correlation of 0.02 to 0.10, so harvest gain or fund capital are the practical primary outcomes for that arm. The intra-cluster correlations are assumptions; the pilot's own data would replace them.

8 Limitations

All farmer data are synthetic, and behavioural parameters — shock sizes, the response of default to type, diversion and capacity, leakage by custody model — are assumptions, bounded but not estimated. Harvest gain is valued output, not profit: family labour and input costs beyond the loan are not netted out. Landholding data are from 2005/06; the urea price behind the naive return scenario comes from an unverified source and is used only as an upper bound. Testnet behaviour was verified for the amendments enabled on 2 October 2026 and may change. Nothing here describes the internals of any deployed system.

9 Conclusion

On the XRP Ledger a spending restriction is only as strong as the custody of the farmer's key, and a vendor willing to collude defeats any ledger rule. Within the model, what the ledger offers a lender is not surveillance of spending but a shared, tamper-evident repayment record — worth most where lending is hardest, and worth little if the underlying return to inputs is low. Which of these hold for real farmers is a question for a field trial, and the simulation says which trial would be informative.

References

J. Chamberlin. *It's a Small World After All: Defining Smallholder Agriculture in Ghana.* IFPRI Discussion Paper 00823, 2008.

Center for Technology, Economics and Development, NYU Abu Dhabi. Blockchain Research. `https://www.nyucted.org/blockchain-research`, accessed 2 October 2026.

S. M. Ichami, K. D. Shepherd, A. M. Sila, J. J. Stoorvogel, E. Hoffland. Fertilizer response and nitrogen use efficiency in African smallholder maize farms. *Nutrient Cycling in Agroecosystems* 113(1):1–19, 2019.

D. Karlan, E. Kutsoati, M. McMillan, C. Udry. Crop price indemnified loans for farmers: a pilot experiment in rural Ghana. Working paper, November 2010.

D. Karlan, R. Osei, I. Osei-Akoto, C. Udry. Agricultural decisions after relaxing credit and risk constraints. *Quarterly Journal of Economics* 129(2):597–652, 2014.

Ministry of Food and Agriculture, Statistics, Research and Information Directorate. *Agriculture in Ghana: Facts and Figures 2024.*

NYU Abu Dhabi. NYU Abu Dhabi receives generous grant for blockchain research. News release, August 2026. `https://nyuad.nyu.edu/en/news/latest-news/community-life/2026/august/blockchain-research-grant.html`, accessed 2 October 2026.

P. Virtanen et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods* 17:261–272, 2020.

XRP Ledger documentation. Deposit Authorization. `https://xrpl.org/docs/concepts/accounts/depositauth`, accessed 2 October 2026.

XRP Ledger documentation. Permissioned Domains. `https://xrpl.org/docs/concepts/tokens/decentralized-exchange/permissioned-domains`, accessed 2 October 2026.